

vLLM 스케줄러 개선

SJF 기반 스케줄링 구현

모델: Llama3-8B | 예측기: OPT-125m | GPU: H100 80GB

01

배경 및 구조 변경

기존 시스템의 한계와 개선 방향

02

실험 설계

Request Rate 실험 · Concurrency 실험

03

실험 결과

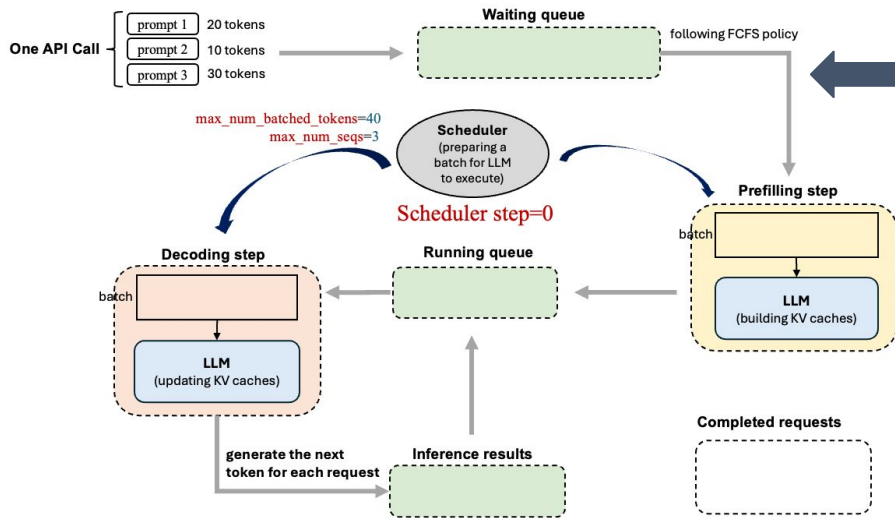
TTFT · TPOT · E2E 비교

04

결론

SJF 효과 발현 조건 및 trade-off

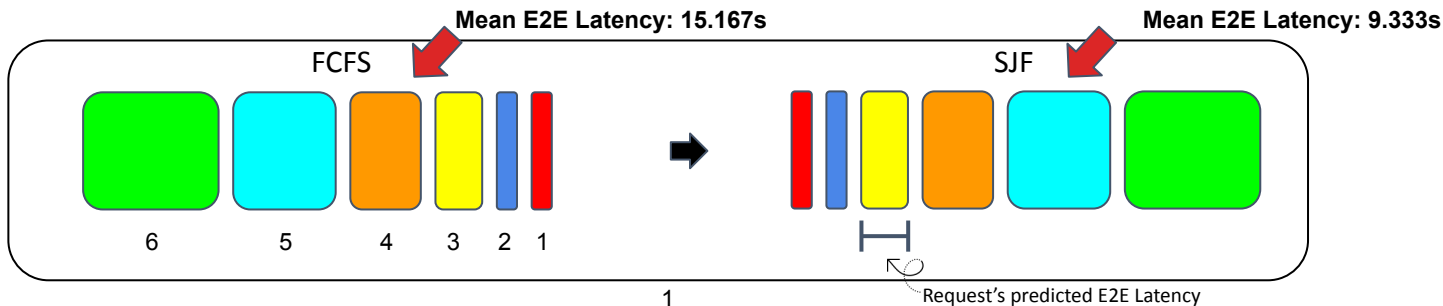
01 배경 및 구조 변경



vLLM's default Scheduling Policy:
FCFS (First Come First Service)

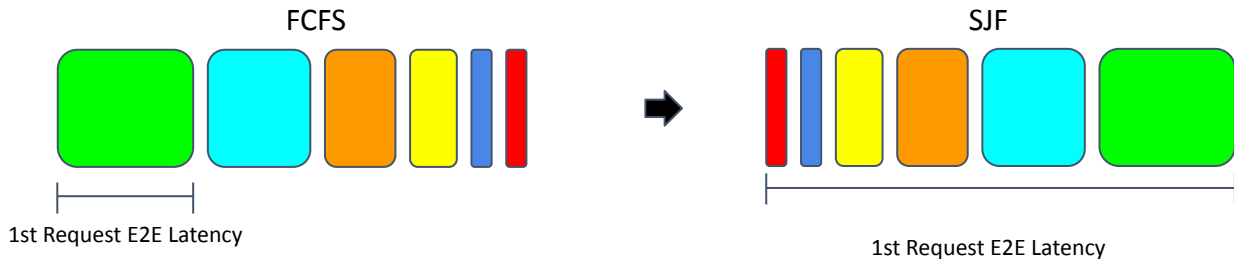
평균 지연시간 감소하기 위해 Batch에 넣기
이전 과정에서 FCFS (First Come First Service)
가 아닌,
SJF (Shortest Job First)를 적용해 요청

SJF의 우선순위는 요청에 대한 E2E Latency
순위 예측을 통해 결정



01 배경 및 구조 변경

Problem of Shortest Job First: Starvation



As certain request's E2E Latency can decrease,
other request's E2E Latency can increase
→ worst case: 특정 요청의 E2E Latency 매우 증가

정렬 기준

$$\text{score} = - (\text{base_score} - \text{AGING_FACTOR} \times \text{wait_time})$$

02 Input Distribution (ShareGPT V3)

Prompt Length (tokens)

Seed	Min	Q1	Q2	Q3	Max	Avg	Std
42	4	21	115	406	1020	241.4	264.5
123	4	20	88	369	1014	221.5	255.3
456	4	20	93	394	1014	230.4	264.5
전체	4	20	96	390	1020	231.1	261.6

Output Length (tokens)

Seed	Min	Q1	Q2	Q3	Max	Avg	Std
42	4	24	134	342	999	213.4	227.4
123	4	25	124	321	1008	203.5	215.7
456	4	28	134	312	996	203.5	209.3
전체	4	26	131	325	1008	206.8	217.7

분포 특성

SJF는 Output 길이 예측에 따라 우선순위를 부여하므로, Output 길이 분포가 다양할수록 정렬의 효과가 부각됨

실제 사용한 데이터는 Output 길이 기준 하위 25% = ~25 Token, 상위 25% = ~320 Token에 달하며, 최대 약 1000 Token까지 분포하는 long-tail 특성을 지님

→ 짧은 요청과 긴 요청이 혼재하는 환경에서 SJF 효과가 잘 나타남

실험 신뢰성 높이기 위해 Random Seed 3개로 진행

03 실험 설계

공통 설정

요청 수: 1000 | 반복: 3회 (seed: 42, 123, 456) | Warm-up: 10회 | max_model_len: 2048

실험 1: Request Rate

목적: 실제 서빙 환경 시뮬레이션

Request Rate: 0 ~ 100 req/s

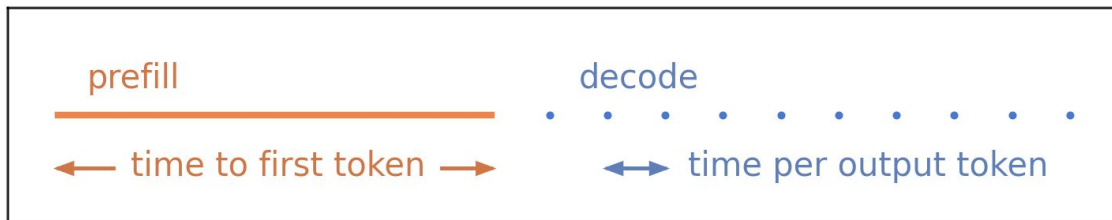
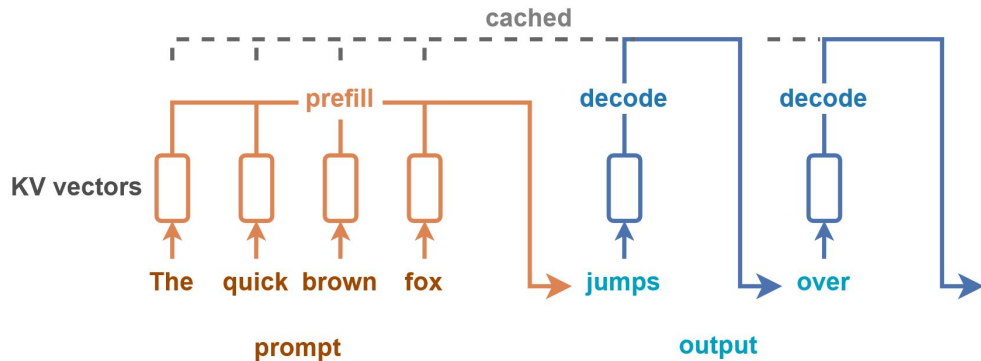
실험 2: Concurrency

목적: SJF 알고리즘 효과 직접 측정

Concurrency: 10 ~ 100

Rate는 시간에 따라 Concurrency가 변동 / Concurrency 실험은 처음부터 끝까지 일정한 queue 경쟁 상태 유지

Preliminary: LLM



Elapsed Time

<https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests>

04 실험 1: Request Rate 0~100 결과

Rate	TTFT avg	TTFT p99	TPOT avg	TPOT p99	E2E avg	E2E p99
1~70	~0%	~0%	~0%	~0%	~0%	~0%
80	▼0.4%	▲10.6%	~0%	▲1.3%	▲0.2%	▼0.2%
90	▼3.0%	▼1.6%	▼0.6%	▼3.1%	▼0.8%	▼1.3%
100	▼3.7%	▲7.4%	▲0.5%	~0%	▲0.2%	▲0.3%
0	▼33.4%	▼4.0%	▲18.5%	▼5.3%	▼0.6%	▲1.0%

Request Rate가 증가할수록 TTFT avg 개선 및 TPOT avg 악화

E2E 유지됨

04 실험 2: Concurrency 10~1000결과

Rate	TTFT avg	TTFT p99	TPOT avg	TPOT p99	E2E avg	E2E p99
10	▲0.5%	▼4.9%	~0%	~0%	▼0.1%	▲0.5%
50	▲0.5%	▲0.8%	▲1.3%	▼0.9%	▼0.3%	▼0.3%
100	▲0.8%	▼5.8%	▲1.0%	▲13.7%	~0%	▲0.3%
200	▼14.5%	▼4.3%	▲4.3%	▲12.7%	▼0.3%	▼1.9%
500	▼29.1%	▲5.6%	▲13.5%	▲0.3%	▼0.3%	~0.0%
1000	▼31.1%	▲3.6%	▲17.1%	▼1.6%	▲1.2%	▲1.3%

Concurrency Rate (동시에 처리하는 요청 수)가 많을수록 TTFT avg 개선과 동시에 TPOT avg 악화

TTFT p99와 TPOT p99: 일관된 개선 보이지 않음

Shortest Job First로 스케줄링한 경우 Starvation 발생하여 E2E 악화되었을 가능성 존재함.
→ Threshold 적용

05 MAX_WAIT Threshold 적용

```
MAX_WAIT_SECONDS = 10.0

def pop_request(self):
    now = time.time()
    # MAX_WAIT 초과 요청 → 강제 우선 처리
    for sorter in self._heap:
        if now - sorter._arrival_time > self.MAX_WAIT_SECONDS:
            return sorter.request
```

동작 방식

pop_request() 호출 시 heap
전체를 순회

대기 시간이
MAX_WAIT_SECONDS(10초)
초과한 요청 발견 시 즉시 반환

없으면 기존 SJF 정렬(heapify) 후
정상 처리

06 추가 실험 : Threshold 적용 결과

Concurrency별 성능 요약

Concurrency	TTFT avg (vs SJF)	TTFT p99 (vs SJF)
10	▼0.3%	▲2.5%
50	▼1.3%	▼3.6%
100	▼2.6%	▲3.5%
200	▲5.1%	▲0.3%
500	▼3.3%	▼0.4%
1000	▼4.9%	▼1.3%

TTFT AVG: concurrency=500~1000에서 3~5% 개선

TTFT p99: 일관된 개선 보이지 않음

TPOT & E2E: 기존 SJF와 동일

07 실제 TPOT 악화 원인: KV Cache 사용률

시간대 별 KV Cache Block 사용량 (percentage of vLLM's allocated GPU Mem)

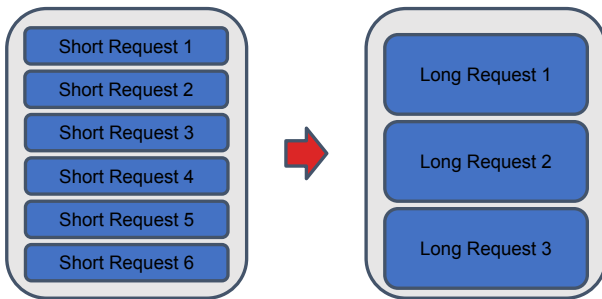
Elapsed	FCFS KV%	SJF KV%
12s	3.4%	3.0%
14s	20.6%	19.6%
16s	30.6%	28.6%
17s	28.6%	34.7%
18s	25.9%	27.0%
21s	28.6%	27.9%
25s	17.6%	19.4%

SJF: 짧은 요청 우선 처리 → 후반에 긴 요청 몰림
→ 긴 요청일 수록 KV Cache에 메모리 더 사용함

vLLM의 Continuous Batching에 따르면 Batch에 들어갈 수 있는 요청 수는 (허용 메모리 용량) / (각 요청 차지하는 메모리 용량)

시간이 지날수록 SJF의 Batch 당 Request 수는 감소하고, GPU 활용률 감소 (긴 요청에 대해 Batch 수 증가)

KV Cache Block Size가 커 지면 시간이 긴 요청들의 GPU 호출 횟수 증가하여 평균 TPOT 증가



08 종합 결론

	Rate 실험	Concurrency 실험
SJF 효과 발현 조건	Request Rate ≥ 80 req/s	Concurrency ≥ 200
최대 TTFT avg 개선	▼33.4% (rate=0)	▼31.1% (conc=1000)
E2E 변화	~0%	~0%
TPOT trade-off	-18.5%	-17.1%

SJF는 TTFT avg/p50을 최대 30~47% 개선 (고부하 조건 한정)

TPOT ~17~19% 악화 — 짧은 요청 우선 처리로 인한 예상된 trade-off (긴 요청만 있을 경우 요청 당 사용하는 KV Cache가 증가하고, 이로 인한 GPU 활용률 저하)

E2E latency는 전 구간에서 거의 동일 (두 효과 상쇄)

첫 토큰 응답성이 중요한 chatbot 서빙 + 고부하 환경에서 특히 효과적
혹은 Decoding이 없어 TPOT의 trade-off가 해결된 Prefill-Only Workload에서 성능 향상이 될 것으로 예상됨